

✿ Ejercicio: Análisis de notas de estudiantes

Objetivo

Practicar el uso del *debugger* de VS Code para detectar errores lógicos en un programa que usa listas y diccionarios.

Contexto

El siguiente código intenta calcular el promedio de notas de varios estudiantes y mostrar al mejor alumno.

Sin embargo, contiene varios *bugs* que deberán detectarse usando el *debugger* (no “a ojo”).

Código (con errores)

analizar_notas.py

```
estudiantes = [
    {"nombre": "Ana", "notas": [8, 7, 9]},
    {"nombre": "Luis", "notas": [5, 6]},
    {"nombre": "Marta", "notas": [10, 9, 10]},
    {"nombre": "Pedro", "notas": []}
]

def calcular_promedio(notas):
    total = 0
    for n in notas:
        total += n
    promedio = total / len(notas)
    return promedio

def mejor_estudiante(lista):
    mejor = None
    mejor_promedio = 0

    for est in lista:
        promedio = calcular_promedio(est["notas"])
        if promedio > mejor_promedio:
            mejor = est["nombre"]
            mejor_promedio = promedio
    return mejor, mejor_promedio

print("Calculando promedios...\n")

for est in estudiantes:
    print(est["nombre"], "=> promedio:", calcular_promedio(est["notas"]))

nombre, nota = mejor_estudiante(estudiantes)
print(f"\nEl mejor estudiante es {nombre} con una nota media de {nota:.2f}")
```

Ejecuta una vez sin depurar

Pulsa Ctrl+F5 o usa el menú:

Ejecutar → Ejecutar sin depuración

Deberías ver el error `ZeroDivisionError` en la terminal integrada.

Eso confirma que el archivo se ejecuta correctamente.

Configura el depurador (opcional, pero recomendable)

Ve a Run and Debug/créate a launch.json file y añade Python debugger/Python File y sale un archivo launch.json con este contenido:

```
{
  // Use IntelliSense to learn about possible attributes.
  // Hover to view descriptions of existing attributes.
  // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
  "version": "0.2.0",
  "configurations": [

    {
      "name": "Python Debugger: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal"
    }
  ]
}
```

Si quiero FIJAR el archivo:

```
"program": "${workspaceFolder}/analizar_notas.py",
```

Checklist

- Ejecutar con y sin depuración.
- Poner y quitar breakpoints.
- Usar **Step Into (F11)** / **Step Over (F10)** / **Continue (F5)**.
- Leer **Variables** y **Pila de llamadas**.
- Añadir **Watch**.
- Configurar **Breakpoint condicional**.
- Crear **Logpoints**.
- Marcar **Detener en excepciones** y entender dónde falló.

RETOS

A) Corrige el código para que no falle con try/except

B) Solo diccionarios

- Cambia la estructura a:
- `estudiantes = {`
- `1: {"nombre": "Ana", "notas": [8,7,9]},`
- `2: {"nombre": "Luis", "notas": [5,6]},`
- `3: {"nombre": "Marta", "notas": [10,9,10]},`
- `4: {"nombre": "Pedro", "notas": []}`
- `}`

y recórrelo con:

```
for id_est, est in estudiantes.items():
```

```
...
```

Reto: Breakpoint condicional cuando `id_est == 4` o `len(est["notas"]) == 0`.

C) Tipos mezclados (bug clásico)

- Inserta un valor no numérico:
- `{"nombre": "Luis", "notas": [5, "6"]}`

Reto: Detén la ejecución donde `n` no sea `int/float`; usa **condición** `not isinstance(n, (int, float))`.

D) Logpoints útiles

- Añade logpoints en cada iteración del for dentro de `calcular_promedio`:
 - Mensaje: Sumando `n={n}`; `total={total}`